Problem Set 3

Peter Shi, Wenhui Xu

June 21, 2026

Question 1: Interpolation and Extrapolation for Utility Functions

This question studies interpolation accuracy and computational cost for three utility functions:

$$u(c) = \log(c), \qquad u(c) = \sqrt{c}, \qquad u(c) = \frac{c^{1-\alpha}}{1-\alpha} \text{ for } \alpha = 2, 5, 10.$$

We work on the domain $[a, b] = [0.10, 5.0]$. For each function we construct: (i) linear interpolation, and (ii) cubic spline interpolation. Accuracy is judged by evaluating the interpolant on an equally spaced dense grid with increment 0.01 over $[0.10, 5.0]$, and checking for visible oscillations (especially near the lower bound where curvature is high). We also compute a simple max absolute error on the dense grid for reference.

A key constraint in this homework is that the interpolation method should not be a black box: we must be able to choose grid/knot locations (including expanding grids) and specify boundary conditions for spline interpolation.

Preliminaries: Utility and derivatives

For spline interpolation, we use clamped (endpoint-slope) boundary conditions with the *true derivatives* at the endpoints:

$$u'(a), \quad u'(b).$$

These are computed analytically for each utility.

```
# Utilities and true derivatives

u_log(c) = log(c)
du_log(c) = 1 / c

u_sqrt(c) = sqrt(c)
du_sqrt(c) = 1 / (2 * sqrt(c))

u_crra(c, α) = (c^(1-α)) / (1-α)
du_crra(c, α) = c^(-α)
```

(a) Linear vs cubic spline interpolation with expanding grids

Grid design: expanding grids

To reduce computational burden, we do not necessarily use an equally spaced grid in c. Instead, we use an expanding grid controlled by an exponent θ:

$$c_i = a + (b-a)\left(\frac{i-1}{N-1}\right)^{\theta}, \quad i = 1, \ldots, N.$$

When $\theta > 1$, more points are concentrated near the lower bound ($c \approx a$), which is useful for utility functions because curvature is highest near the lower end.

```
function expanding_grid(a, b, N, theta)
    t = range(0.0, 1.0; length=N)
    return a .+ (b-a) .* (t .^ theta)
end
```

Interpolation routine (one run)

For a given utility u and derivative u', we: 1) generate the expanding grid x and function values $y = u(x)$, 2) build a linear interpolant, 3) build a clamped cubic spline interpolant using endpoint slopes $u'(a), u'(b)$, 4) evaluate both interpolants on a dense grid with step 0.01, 5) plot true vs interpolants, and plot a zoom near the lower end to check oscillations.

```
using Interpolations
using BasicInterpolators
using Plots

function interplants(u, du; a=0.10, b=5.0, N=40, theta=1.0, zoom_max=0.30, label
    ="")
    x = expanding_grid(a, b, N, theta)
    y = u.(x)

    dense = collect(a:0.01:b)       # 0.01 increment grid
    ytrue = u.(dense)

    # (i) linear interpolation
    lin = LinearInterpolation(collect(x), collect(y), extrapolation_bc=Throw())
    y_lin = lin.(dense)

    # (ii) cubic spline with clamped endpoint slopes
    s_left  = du(a)
    s_right = du(b)
    spl = CubicSplineInterpolator(collect(x), collect(y), s_left, s_right)
    y_spl = spl.(dense)

    err_lin = maximum(abs.(y_lin .- ytrue))
    err_spl = maximum(abs.(y_spl .- ytrue))
```

```
    p_full = plot(dense, ytrue; label="true", xlabel="c", ylabel="u(c)",
                  title="Full: $label (N=$N, theta=$theta)")
    plot!(p_full, dense, y_lin; label="linear (max err=$(round(err_lin,
        sigdigits=3)))")
    plot!(p_full, dense, y_spl; label="cubic spline clamped (max err=$(round(
        err_spl, sigdigits=3)))")

    idx = findall(<=(zoom_max), dense)
    p_zoom = plot(dense[idx], ytrue[idx]; label="true", xlabel="c", ylabel="u(c)
        ",
                  title="Zoom low end (<= $zoom_max): $label (N=$N, theta=$theta
                      )")
    plot!(p_zoom, dense[idx], y_lin[idx]; label="linear")
    plot!(p_zoom, dense[idx], y_spl[idx]; label="cubic spline clamped")

    return (p_full=p_full, p_zoom=p_zoom, err_lin=err_lin, err_spl=err_spl,
            s_left=s_left, s_right=s_right, xgrid=x)
end
```

Choosing θ and N (minimize function evaluations subject to accuracy)

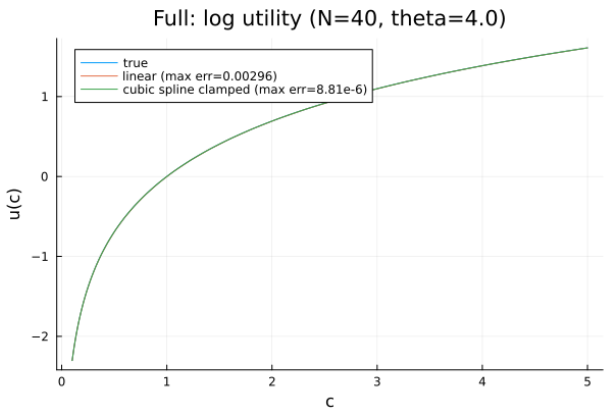
The objective is to use as few function evaluations as possible (i.e., small N), while still producing an accurate interpolant on off-grid points (no oscillations under zoom).
We tested $\theta \in \{1, 1.5, 3, 4\}$ and increased N until the interpolated curves overlap the true function on the dense grid, including a zoomed plot near the lower end.
Based on the zoom-in checks and our comments in the notebook, the chosen "minimal N with no visible wiggles" configurations are summarized in Table 1.

Table 1: Chosen expanding-grid settings (minimal N subject to accurate, no-wiggle interpolation)

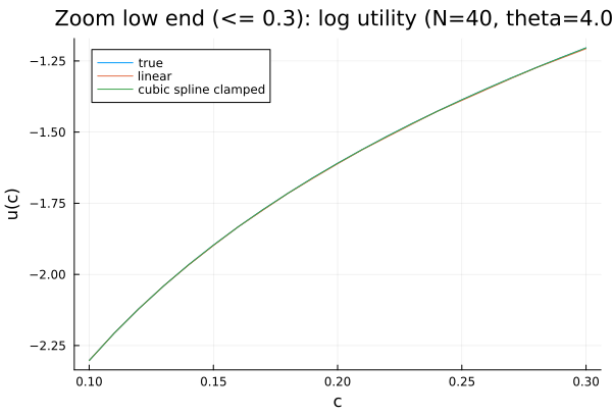
Utility	θ	Chosen N	Notes (from zoom checks)
$\log(c)$	4.0	40	$\theta = 4$ gave clean overlap with fewer points than $\theta = 3$
$\sqrt{c}$	4.0	20	larger θ allowed much smaller N than $\theta = 1$
CRRA, $\alpha = 2$	4.0	45	higher curvature near a required more points
CRRA, $\alpha = 5$	4.0	80	stronger curvature $\Rightarrow$ higher N
CRRA, $\alpha = 10$	4.0	80	strongest curvature among tested CRRA cases

Figures for Q1(a)

Insert the plots you generated in the notebook here. The main requirement is that the figure is informative and that the zoom reveals whether there are oscillations.

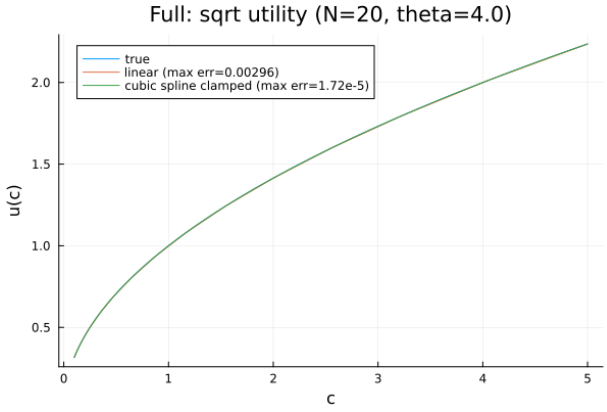


(a) $\log(c)$ full

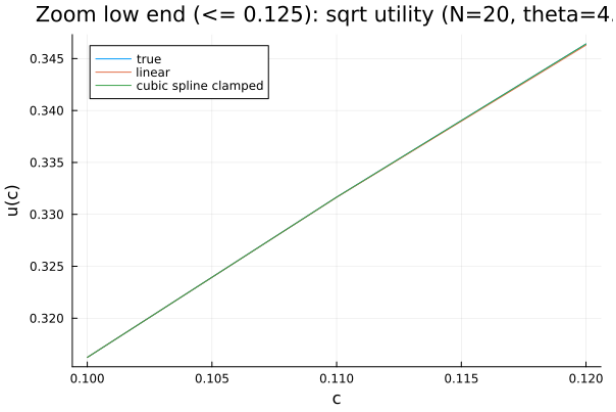


(b) $\log(c)$ zoom (low end)

Figure 1: log utility interpolation (linear vs clamped cubic spline) on dense grid with 0.01 increments.

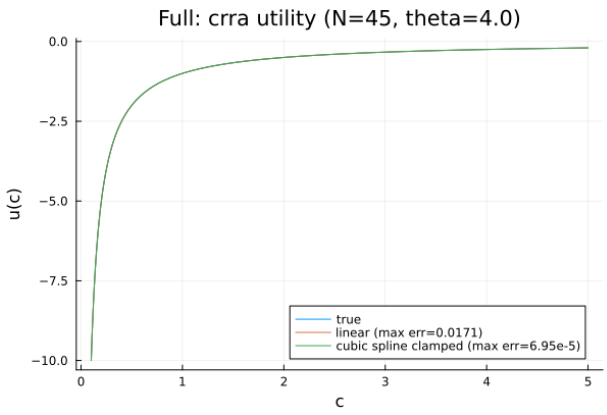


(a) $\sqrt{c}$ full

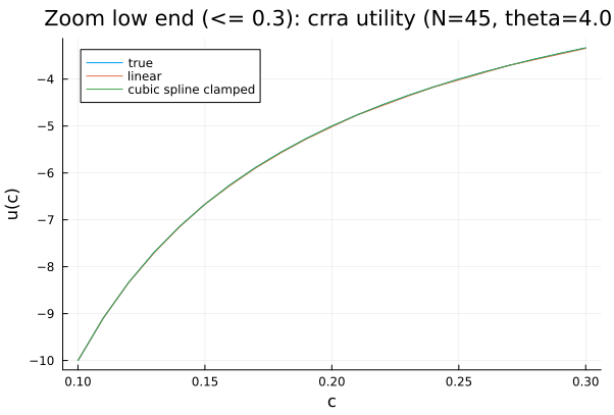


(b) $\sqrt{c}$ zoom (low end)

Figure 2: sqrt utility interpolation (linear vs clamped cubic spline) on dense grid with 0.01 increments.

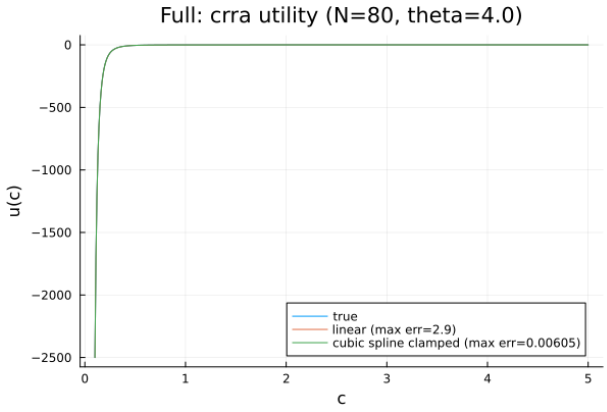


(a) $\log(c)$ full

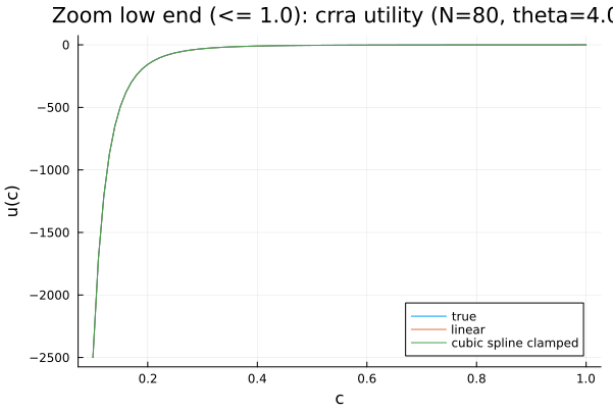


(b) $\log(c)$ zoom (low end)

Figure 3: CRRA($\alpha = 2$) utility interpolation (linear vs clamped cubic spline) on dense grid with 0.01 increments.

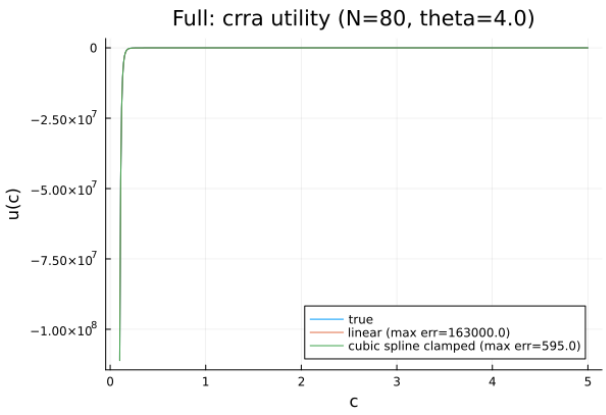


(a) $\log(c)$ full

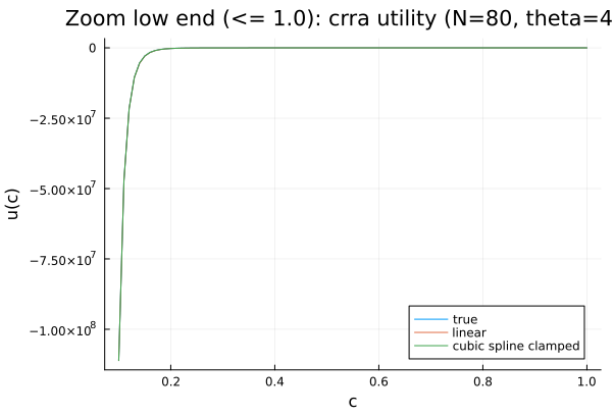


(b) $\log(c)$ zoom (low end)

Figure 4: CRRA($\alpha = 5$) utility interpolation (linear vs clamped cubic spline) on dense grid with 0.01 increments.



(a) log(c) full



(b) log(c) zoom (low end)

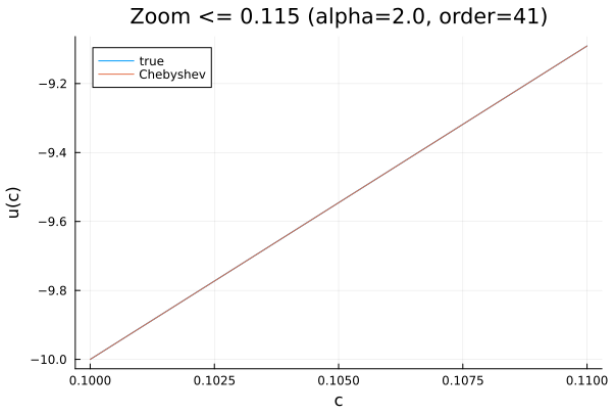
Figure 5: CRRA($\alpha = 10$) utility interpolation (linear vs clamped cubic spline) on dense grid with 0.01 increments.

(b) Chebyshev polynomial interpolation for CRRA ($\alpha = 2, 10$)

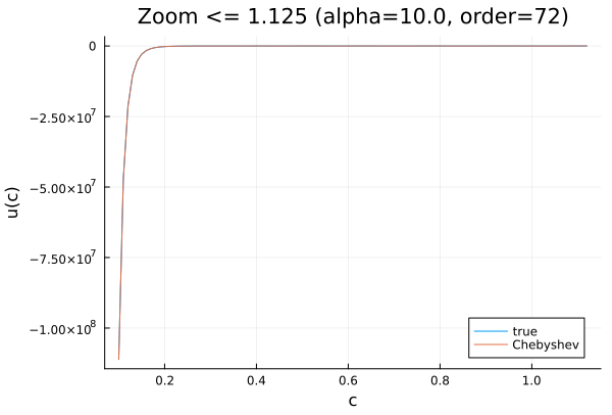
In part (b) we use Chebyshev polynomial interpolation for CRRA with $\alpha = 2$ and $\alpha = 10$. Chebyshev methods use special node placements (Chebyshev nodes) that reduce the Runge phenomenon in the interpolation *inside the domain*. However, because the interpolant is global, it can be sensitive near boundaries and can behave very poorly under extrapolation. In our experiments, the minimal orders that produced accurate interpolation with no visible wiggles (under zoom) were:

$$\alpha = 2: \ p = 41, \qquad \alpha = 10: \ p = 72.$$

As curvature increases (larger α), a higher order is typically needed to maintain accuracy near the lower bound.



(a) log(c) full



(b) log(c) zoom (low end)

Figure 6: Chebyshev polynomial interpolation for CRRA($\alpha = 2, 5$) with minimal accurate polynomial orders.

(c) The perils of extrapolation

We evaluate each interpolation scheme at the out-of-domain points:

$$c \in \{0.05,\ 5.1,\ 5.5,\ 6.0\}.$$

We consider three schemes: (i) linear interpolation with linear extrapolation enabled, (ii) clamped cubic spline inside $[a, b]$ and *tangent-line extrapolation* outside using endpoint derivatives, (iii) Chebyshev polynomial (global polynomial) evaluation outside the domain.

What does "feasible" mean here?

A method is *feasible* for extrapolation if it returns a numerical value without throwing an error and without producing obviously undefined outputs (e.g., NaN). In practice, many interpolation packages intentionally forbid out-of-domain evaluation for safety; this is particularly common for Chebyshev implementations.

What does "better" mean here?

Because the true utility function is known at these test points, we define "better" as: smaller absolute extrapolation error at the test points, and no catastrophic blow-ups that violate the expected magnitude/shape of the utility function.

Results: log utility

From the notebook output:

Table 2: Extrapolation results for $u(c) = \log(c)$

c	True	Linear	Spline (tangent)	Chebyshev
0.05	-2.9957	-2.8026	-2.8026	-2.9895
5.1	1.6292	1.6305	1.6294	1.6326
5.5	1.7047	1.7146	1.7094	9585.8557
6.0	1.7918	1.8198	1.8094	3.2244×10^8

For log utility, linear and spline-based extrapolation remain well-behaved, with small errors for $c = 5.1$ and moderate errors at $c = 0.05$ (since $c = 0.05$ is not too far below $a = 0.10$ but curvature is high near zero). In contrast, Chebyshev extrapolation becomes unstable for $c = 5.5$ and $c = 6.0$, producing explosive values far outside the scale of $\log(c)$.

Results: square-root utility

Table 3: Extrapolation results for $u(c) = \sqrt{c}$

c	True	Linear	Spline (tangent)	Chebyshev
0.05	0.2236	0.2372	0.2372	0.2238
5.1	2.2583	2.2590	2.2584	2.2584
5.5	2.3452	2.3507	2.3479	368.1808
6.0	2.4495	2.4654	2.4597	1.2310×10^{7}

Again, linear and spline-based extrapolation remain stable and close to the truth, while Chebyshev extrapolation can blow up sharply outside the interpolation domain.

Results: CRRA utility

We report the three CRRA cases using the "optimal grid" choices from part (a), and Chebyshev orders from part (b) (for $\alpha = 2$ and $\alpha = 10$; for $\alpha = 5$ we used the notebook setting $p = 55$).

Table 4: Extrapolation results for CRRA utility (selected entries)

α	c	True	Linear	Spline (tangent)	Chebyshev
2	0.05	-20.0000	-14.9999	-15.0000	-19.5330
2	6.0	-0.1667	-0.1562	-0.1600	1.8754×10^{10}
5	0.05	-40000.0000	-7499.9843	-7500.0000	-31647.7194
5	6.0	-0.0001929	-0.0000369	-0.0000800	2.1036×10^{18}
10	0.05	-5.6889×10^{10}	-6.1111×10^{8}	-6.1111×10^{8}	-2.1227×10^{10}
10	6.0	-1.1025×10^{-8}	7.5631×10^{-8}	4.5511×10^{-8}	-3.0232×10^{29}

Two patterns stand out. First, extrapolating below $a = 0.10$ is difficult for CRRA when α is large, because the true utility diverges rapidly as $c \to 0$; even linear and spline tangent extrapolation can incur very large absolute errors at $c = 0.05$. Second, Chebyshev extrapolation is consistently unsafe: it frequently produces astronomically large magnitudes for $c > 5.0$, which is clearly not meaningful for CRRA utilities.

Conclusion for extrapolation

Across all utilities, linear extrapolation and tangent-line spline extrapolation are feasible and remain numerically stable for the tested points. Between these two, the spline tangent rule tends to deliver slightly smaller errors at the upper out-of-domain points, while both can struggle when extrapolating toward the singular region (e.g., CRRA with large α at $c = 0.05$). Chebyshev extrapolation is not reliable: even when the interpolation inside the domain is accurate, the global polynomial can behave extremely poorly outside $[0.10, 5.0]$.

Question 2

We solve the stochastic income fluctuation problem

$$V(k,z) = \max_{k' \in K} \left\{ u(c) + \beta \, \mathbb{E}\left[V(k',z') \mid z \right] \right\},$$

subject to the budget constraint

$$c + k' = (1+r)k + z,$$

and the AR(1) income process

$$z' = \rho z + \eta, \quad \eta \sim N(0, \sigma_\eta^2),$$

with parameters $\rho = 0.98$, $\sigma_\eta = 0.20$, $\beta = 0.96$, and $r = 0.03$. We use log utility $u(c) = \log(c)$ with the feasibility restriction $c > 0$.

(a) Discretizing the AR(1) with Rouwenhorst and verifying moments

We discretize the AR(1) process using the Rouwenhorst method with $N_z = 29$ states and simulate a long series with $T = 20{,}000$ observations. We verify the discretization by reporting: (i) the first-order autocorrelation of the simulated series; and (ii) the implied standard deviation of the innovation obtained by fitting an AR(1) regression to the simulated series.

```
Nz = 29
ρ  = 0.98
σ_η = 0.20
T = 20000   # > 10,000

zgrid, P = rouwenhorst(Nz, ρ, σ_η)
zsim = simulate_markov(P, zgrid, T)

ρ_est = cor(zsim[2:end], zsim[1:end-1])
println("Autocorrelation: ", ρ_est)

# Fit AR(1): z_t = a + b z_{t-1} + e_t
X = [ones(T-1) zsim[1:end-1]]
y = zsim[2:end]
b = X \ y
resid = y - X*b
println("Implied sigma_eta (std of residual): ", std(resid))
```

From the notebook output, we obtain:

$$\widehat{\rho} = 0.9798673593, \qquad \widehat{\sigma}_\eta = 0.1985193122.$$

Both values are close to their targets $\rho = 0.98$ and $\sigma_\eta = 0.20$, indicating that the Rouwenhorst discretization provides a good approximation to the continuous AR(1) process for our purposes.

(b) Solving the model using the Endogenous Grid Method (EGM)

We solve the dynamic programming problem using the Endogenous Grid Method (EGM). The core idea of EGM is to avoid an explicit maximization over k'. Instead, we: (i) compute expected marginal utility on the exogenous future-capital grid; (ii) use the Euler equation to recover current consumption consistent with each k' on that grid; (iii) construct an endogenous grid for current capital k implied by the budget constraint; and (iv) interpolate the resulting consumption function back onto the original k-grid using cubic splines (as required).
We use a stopping rule based on convergence of the decision rule, with tolerance 10^{-7}.

```
function egm_solver(; β=0.96, r=0.03, ρ=0.98, σ_η=0.20,
                    Nk=200, Nz=29,
                    k_low=0.01, k_high=20.0,
                    tol=1e-7, maxit=2000,
                    verbose=true)

    kgrid = kgrid_default(; Nk=Nk, k_low=k_low, k_high=k_high)
    zgrid, P = rouwenhorst(Nz, ρ, σ_η)

    c = ones(Nk, Nz)
    c_new = similar(c)

    pol_idx = ones(Int, Nk, Nz)
    pol_idx_new = similar(pol_idx)

    it = 0
    converged = false

    while it < maxit
        it += 1

        for iz in 1:Nz
            z = zgrid[iz]

            Emu = zeros(Nk)
            for izp in 1:Nz
                Emu .+= P[iz, izp] .* (1.0 ./ c[:, izp])
            end

            c_today = 1.0 ./ (β * (1 + r) .* Emu)
            m = kgrid .+ c_today
            k_endo = (m .- z) ./ (1 + r)

            sidx = sortperm(k_endo)
            k_endo_s = k_endo[sidx]
            c_today_s = c_today[sidx]

            spl = Spline1D(k_endo_s, c_today_s; k=3, bc="extrapolate")
            c_new[:, iz] .= max.(spl.(kgrid), 1e-12)

            kprime = (1 + r) .* kgrid .+ z .- c_new[:, iz]
            for ik in 1:Nk
                pol_idx_new[ik, iz] = argmin(abs.(kgrid .- kprime[ik]))
            end
```

```
        end

        g_old = kgrid[pol_idx]
        g_new = kgrid[pol_idx_new]
        dist = maximum(abs.(g_new .- g_old))

        c .= c_new
        pol_idx .= pol_idx_new

        if verbose && (it % 25 == 0)
            println("EGM it=$it, dist=$dist")
        end

        if dist <= tol
            converged = true
            break
        end
    end

    g = kgrid[pol_idx]
    return (; c, g, kgrid, zgrid, P, it, converged)
end
```

In our benchmark output, EGM converges in 94 iterations (`converged = true`).

(c) Plotting decision rules and value functions

Figure 7 reports the savings decision rule $k'(k, z)$ and the associated consumption rule $c(k, z)$ across a subset of income states z.

The left panel shows that $k'(k, z)$ is increasing in k for each fixed z, and higher income states shift the policy upward, which is economically intuitive: when the shock is high, resources are higher and the agent can save more for the next period. The (approximate) step-like appearance is expected when a continuous choice is mapped back to the nearest point on the discrete grid.

The middle panel plots $c(k, z)$. In our implementation, consumption is forced to remain strictly positive (via `max(., 1e-12)`), so when the implied consumption from the spline interpolation is extremely small, the curve can appear "stuck" close to zero on the plot scale. Occasional spikes or jagged segments near the lower end can occur because (i) the budget constraint is tight at low k, (ii) the interpolation is performed on an endogenous grid and then mapped back to the exogenous grid, and (iii) the discrete nearest-grid mapping for k' can create small discontinuities. These patterns are numerical artifacts rather than an intended economic feature.

EGM does not automatically return $V(k, z)$ in the same way as VFI. Therefore, if the right panel is labeled as `proxy: u(c)`, it plots $u(c(k, z))$ as a shape proxy. When policy evaluation is performed, we instead iterate on

$$V(k, z) = u(c(k, z)) + \beta \sum_{z'} P(z, z')\, V(k'(k, z), z'),$$

holding the converged policy fixed until the value function converges. In our notebook output,

policy evaluation converges for EGM with:

$$\texttt{Vegm.it} = 646, \quad \texttt{Vegm.converged} = \texttt{true}, \quad \texttt{Vegm.dist} = 9.7770680441 \times 10^{-11}.$$

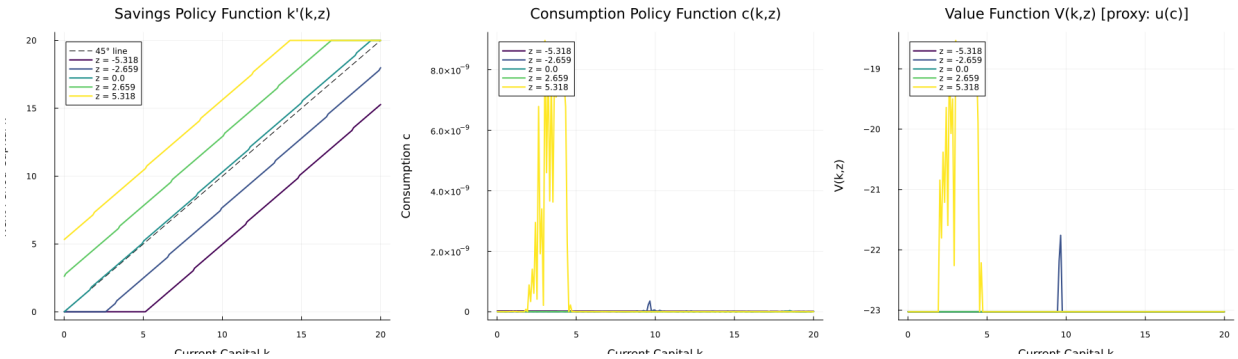


Figure 7: EGM results across selected z states: (left) savings policy $k'(k, z)$, (middle) consumption policy $c(k, z)$, and (right) value function panel. If the right panel is labeled as `proxy: u(c)`, then it plots $u(c(k, z))$ as a proxy rather than the fully evaluated $V(k, z)$.

Runtime comparison (VFI vs. EGM)

We benchmark the two solution methods using `@belapsed` (after a warm-up run to exclude compilation time). EGM converges in fewer iterations and is substantially faster than VFI in this setup.

Table 5: Runtime comparison of VFI and EGM (Julia `@belapsed`)

Method	Iterations (it)	Converged	Time (seconds)
VFI	326	Yes	19.7303
EGM	94	Yes	1.0053

We also report a speedup of approximately

$$\frac{19.7303}{1.0053} \approx 19.6\times,$$

meaning EGM is about 20 times faster than VFI for the same parameterization and grid choices.